# Lab 2: Starting with JavaScript ES6 [JavaScript ES6]

## Learning Outcomes:

- Become comfortable with JavaScript syntax, working with various data types and operators.
- Understand the difference between var, let and const keywords, and impact on scope, in JavaScript ES6.
- Master the use of logic and decision making structures in JS, e.g., conditionals, loops, functions; as well as data structures.

## Instructions:

- For this lab, you will be applying the basics of JavaScript. In particular, you will be working with various data types (e.g., numbers, strings, booleans and arrays) and variables, manipulating data and printing results, logic and decision making structures, and testing your work through the browser's **JS Console**.
- For this lab, you will be asked to create a Class Grade Book, for now you do not have to worry about implementing an HTML form for data entry, you will be providing the data for your grade book within your JS code.
- Your lab should follow proper folder structure (i.e., your script.js file should be stored within a 'js' or 'script' folder, this folder should be nested within your 'lab2' folder, any CSS code should reside in a 'css' folder).
- **The requirements for your Grade Book are:**
    - Your class grade book should be displayed in an HTML page, this page should be named **index.html** and reside directly inside your **'lab2' folder** within your **csci3172 directory** on **Timberlea**. Your grade book **MUST NOT** make use of objects.
    - Your class grade book should display the following of information in a table structure (See Figure 1 for an example):
        - **'*Student Name*':** should only make use of a student's first name. You will be expected to have at least EIGHT (8) students in your grade book.
        - '***Grades':*** as each student is expected to be enrolled in FOUR (4) courses, they will each be expected to have at least FOUR (4) '***course grades'*** assigned to them.
        - ***'Average Grade':*** your grade book should implement a function to calculates and display the '***average grade'*** for each student for the term. Your average grade should use only round numbers (i.e., no decimals)

- Your grade book should also implement a function to calculate, based on the average numeric grade received, the student's average ***'letter grade'*** for the term.

- At least TWO (2) of your students must be missing at least ONE (1) one course grade, i.e., your script must be able to handle missing grades.

**Note:** You may use the Dalhousie Guide for Grade Scale and Definitions (https://www.dal.ca/campus_life/academic-support/grades-and-student-records/grade-scale-and-definitions.html) to help you determine the Letter Grade equivalent for a Numeric Grade.

Figure 1. Grade book Example.

| Student Name | Course1 | Course2 | Course3 | Course4 | Avg. Number Grade | Avg. Letter Grade |
|---|---|---|---|---|---|---|
| Alice | 89 | 79 | 94 | 90 | 88 | A |
| Pedro | 77 | 81 | 89 | | 82 | A- |
| Jeff | 73 | | 71 | 85 | 76 | B |
| Laura | 80 | 91 | 63 | 78 | 78 | B+ |

- Allow the user to click on at least THREE (3) of the student names in your grade book, this action should take the user to a page displaying that student's grades for each assessment type in each course the student was enrolled in:

  - The course has THREE (3) assignments (30% of the final grade), TWO (2) quizzes (20% of the final grade), and TWO (2) exams (50% of the final grade).

  - The user should be able to see the numeric grades received by the student in each assessment, AND the weighted average for each assessment type. However, at least TWO (2) of the students must be missing at least ONE (1) assessment grade (e.g., they received a mark for only one quiz, or they received a grade for 2 assignments instead of 3).

**Note:** The user should be able to navigate back and forth between the pages you have included as part of your lab.

- In addition to displaying your output to the user as part of your HTML page, you are also expected to log all student data to your **JS Console**.

- Your lab must make use of **let** and **const** variable keywords, you may judge the suitability of each keyword based on the plan for your script. In your README file, provide the answers to the following questions:

  **(a)** What is the scope of a **`const`** variable?

  **(b)** When were the **`let`** and **`const`** keywords added to JavaScript?

  **(c)** If you were to change **`let`** for **`var`** in your script(s), would your script still work? Why or Why not?

- Your lab must also make use of **conditional and/or control structures** (e.g., loops, conditionals), you are encouraged to consider the efficiency of your code as your opt for any of these options.

- You have complete creative freedom on the look-and-feel of your webpage, you may also use any CSS framework to design your webpage, as long as its use is customized and documented in your README file. You are encouraged to work towards an aesthetically pleasing website that applies the design and development principles you have learned thus far in your academic and/or web development career. You may use Creative Commons images and/or logos with proper author attribution (provided through your **README.txt** file).

**Note:** You **MAY NOT** use any readily available JS library to help you in the completion of this lab. Do keep in mind that as part of this assignment, you are expected to work individually, you may discuss ideas with your classmates, but do refrain from sharing any code.

- As in previous labs and as specified in the **Submission Section**, for Lab 2 you will be expected to submit a README file, a Git Repo, and a remotely accessible lab on Timberlea. ***See Brightspace Lab 2 module.***

- Ensure you have set the proper file and folder permissions for each file and folder you have uploaded to Timberlea's **public_html** folder as part of your Lab 2 work.

**Note:** For your files to be accessible through a browser for testing and grading, you must ensure you are using the correct file permission settings on your files and folders. On a shared server, such as Bluenose, it is recommended to **use '755' (i.e., rwxr-xr-x) on folders**, and **'644' (i.e., rw-r--r--) on individual files**. You can set your file permissions easily through an FTP client by right clicking on the file or folder you want to set specific permission settings. Depending on your FTP client, you will need to click on **'Get Info'** or **'File Permissions'**. Once on the file permissions window, you can simply enter the numeric value described above.

- Visit **https://web.cs.dal.ca/~yourCSID/csci3172/labs/lab2/** on any browser and ensure you can view your work. If you are not able to view your work, check the location of your files.

**Note:** Failure to submit your work through Timberlea will result in a grade of **ZERO (0).** Failure to ensure your work is remotely accessible through a web browser, using the specified URL will result in a grade of **ZERO (0).** If you can see your work through the specified URL, then the TAs and Instructor will also be able to view and mark it.

- **Include in your README.txt file,** the URL from which your lab can be accessed, all pages you develop for this assignment will need to be accessible through that link. Additionally, make sure you also include the URL from which your Git repo can be accessed.

**Note:** If you decide to use and modify any existing code, e.g., code found on online or printed sources or code used during in-class/tutorial examples, you are expected to provide author attribution in your code comments, and a more detail explanation of your sources in your README file (i.e., providing an explanation of why the piece of code is necessary for your work, where, how and why the code or section of code was

modified). Keep in mind that simply stating "code was modified" does not provide sufficient information required in your programming assignments.

## Submission:

- For this lab, you will need to **submit your work through Timberlea, Brightspace, and GITLab as follows:**

### Submitting your Work through Timberlea

- As part of this lab, you will need to create a **'lab2'** directory inside of your **'csci3172** directory on Timberlea. ***See Lab 1 instructions*** *on how to log onto Timberlea using an FileZilla, and create directories.*

- Once you have completed your lab, upload your work into your **'lab2'** directory on Timberlea.

***Note:*** You will need to ensure your submission includes all required files needed for your Lab 2 (i.e., image files, stylesheets, folders), and that your new directory and individual files have the correct **folder permissions**(i.e., **755**) and **file permissions** applied (i.e., **644**), respectively.

- Your Lab 2 submission will be expected to follow proper folder structure, i.e., images should be inside an 'image', 'images', or 'img' folder, CSS stylesheets should be inside a 'styles' or 'css' folder, and JS scripts should be inside a 'script' or 'js' folder.

- Ensure you have set the proper file and folder permissions for your Lab 2 work.

***Note:*** In order for your files to be accessible through a browser for testing and grading, you must ensure you are using the correct file permission settings on your files and folders. On a shared server, such as Bluenose, it is recommended to **use '755' (i.e., rwxr-xr-x) on folders**, and **'644' (i.e., rw-r--r--) on individual files**. You can set your file permissions easily through an FTP client by right clicking on the file or folder you want to set specific permission settings. Depending on your FTP client, you will need to click on **'Get Info'** or **'File Permissions'**. Once on the file permissions window, you can simply enter the numeric value described above.

- Visit **https://web.cs.dal.ca/~yourcsusername/csci3172/labs/lab2/** on any browser and ensure you can view your work.

***Note:*** Failure to submit your work through Timberlea will result in a grade of **ZERO (0)**. Failure to ensure your work is remotely accessible through a web browser, using the specified URL will result in a grade of **ZERO (0)**.

- No validation is required for this lab as JS cannot be validated as HTML and CSS can be.

- Test your lab to ensure cross-browser compatibility. In this case, you are looking for your functions to be consistent across browsers.

## Submitting your Work through GitLab

- Create a **git** repository on the **FCS Gitlab site**, and clone it to your local system using the following command:

```
git clone *your repo https link*
```

- Copy the HTML or JS file to the local copy of your repo and push it to the git repo using the following commands:

```
git add .
git commit -m "your commit message"
git push
```

- Setup your GITlab repo as a private project and add the course **Teaching Assistants (TAs) and Instructor as 'Maintainers'** to your project, using their **CS IDs.** *See Lab 1 Brightspace module.*

***Note:*** The CSIDs for this course will be provided during our lab session. Failure to add the course CS ID as 'Maintainer' for your work on GitLab will result in a maximum possible grade of 50%.

## Submitting your Work through Brightspace

- Download the **README template** available on Brightspace. *See Resources section on left-hand side menu on Brightspace.* There are TWO versions of this template, you may use whichever you feel more comfortable with.

- Edit the README template to include any citations for your code and/or images used for this Lab.

***Note:*** If the work you are submitting as part of your Lab is work done by you without the use of any external sources, then please specify so within your README file.

- Depending on the version of the template you chose, rename your README file as: **L#_LastName_FirstName_README.txt**

***Note:*** Ensure your README file includes the URL to your Lab for remote access.

# Marking Rubric:

The following grading criteria will be used for marking your lab:

| Dimensions | Does Not Meet Expectations | Somewhat Meets Expectations | Meets Expectations | Exceeds Expectations |
|---|---|---|---|---|
| **Basic Grade Book Functionality (30%)** | Core functionalities of Grade Book (e.g., displaying data, calculating averages, converting to avg. letter grade) are missing or have major errors. **(0 - 7 points)** | Core functionalities are implemented with minor errors (e.g.,inconsistency displaying data, failing to handle cases with missing grades). **(8 - 14 points)** | Core functionalities are implemented with some minor syntax issues or inconsistencies but overall functionality is implemented. **(16 - 22 points)** | Core functionalities are accurately implemented with great attention to detail (e.g., properly displaying data, handling missing grades). **(25 - 30 points)** |
| **Advanced Grade Book Functionality (20%)** | User is not able to view detailed course grades for any student OR grades displayed inconsistently or inaccurately. **(0 - 5 points)** | User is able to view detailed course grades AND weighted avg. for each assessment type for ONE (1) student. **(6 - 10 points)** | User is able to view detailed course grades AND weighted avg. for each assessment type for TWO (2) students. **(11 - 16 points)** | SUser is able to view detailed course grades AND weighted avg. for each assessment type all THREE (3) students. **(18 - 20 points)** |
| **Control and Data Structures (20%)** | Code is difficult to read, shows poor understanding of JS concepts (e.g., variables, data types, control flow, arrays). **(0 - 5 points)** | Code is somewhat readable, use of control and data structures with minor errors, and calculation algorithms applied with some inaccuracies. **(6 - 10 points)** | Code is well-written and readable, appropriate use of control and data structures, and calculation algorithms applied correctly. **(11 - 16 points)** | Code is readable AND efficient, and demonstrates strong understanding of data structures and their application in the lab. **(18 - 20 points)** |
| **JS Console (10%)** | Student's script did not log results on the JS Console. **(0 points)** | | | Student's script logged results on the JS Console. **(10 points)** |
| **Introduction of 'let' and 'const' to JS (10%)** | Student's README file provides no answer. **(0 points)** | Student's README file has somewhat of an answer BUT NO indication of year of introduction or ECMAScript version. **(3 - 5 points)** | Student's README file has a clear answer AND year of introduction BUT NO ECMAScript version stated. **(6 - 8 points)** | Student's README file has a clear answer AND the year of introduction and ECMAScript version is clearly stated. **(10 points)** |
| **Understanding the scope of ''const' (10%)** | Student's README file does not provide an answer. **(0 points)** | Student's README file has somewhat of an answer BUT NO examples to express their understanding. **(3 - 5 points)** | Student's README file includes a clear answer BUT does not provide any examples. **(6 - 8 points)** | Student's README file has a clear answer AND provides examples to further express their understanding. **(10 points)** |
| **Cross-Browser Compatibility & W3C Compliance** | Lab is not cross-browser compatible, noticeable/ distracting differences visible AND lacks W3C compliance. **(-50 points)** | Student's lab is cross-browser compatible BUT lacks W3C compliance. **(-25 points)** | | Student's lab is cross-browser compatible, no noticeable/distracting differences visible AND is W3C compliant. **(0 points)** |
| **Git repository** | Code is not pushed to repo OR Student did not provide Maintainer access to their GitLab repo. **(-50 points)** | | | Code is properly pushed to git repo, AND provided Maintainer access to their Lab's GitLab repo. **(0 points)** |

| | | | | |
|---|---|---|---|---|
| **Timberlea** | Code is not uploaded on Timberlea OR the URL doesn't load the HTML file. **(-100 points)** | | | Code is properly uploaded on Timberlea and URL to file is provided as per requirements. **(0 points)** |
| **README.txt** | Student's did not submit a README.txt file and/or did not edit the template as expected. **(-100 points)** | | Student's submitted a README.txt file that is incomplete or is missing the lab's URL. **(-50 points)** | Student's submitted a README.txt file properly edited to include all sources used. **(0 points)** |